VisIt's Command Line Interface

Introduction

VisIt's command line interface (or CLI) is a Python-based, alternative interface with respect to VisIt's graphical user interface (or GUI). It includes a large set of instructions based on Python 2.7 that allows the user to set the type of plot, its options and to produce images using single instructions or scripts.

Setting up the environment

VisIt is one of the visualization software supported at MPCDF and can be easily accessed using environment modules. For example, to load the latest version of the VisIt module on an interactive node of Hydra, we can just run

```
> module load remotevis
> module load visit/2.10
```

Note that the module for visualization is hidden by default on the interactive nodes of Hydra and we have to unlock it with the first of the previous commands in order to have access to the visualization software.

In order to generate a video from the results, we may also want to load the audio/video codec library using

```
> module load ffmpeg
```

While the visualization software is already provided on Hydra, users may want to install VisIt on their local machine. In order to do so, the executable version of the software can be downloaded for any operating system from this link [1] and installed using the install script provided at the same web page.

Basic commands for the CLI

In this section we will look at some basic commands for VisIt's command line interface. Let's assume we have a database called `elf_rs.vtk` in the current directory. Let's also suppose that it contains a variable called `Re` that we would like to plot. We can run VisIt with the command

```
> visit -cli     <– Run the CLI interface with a plotting window
```

and produce a simple pseudocolor plot by typing in the terminal the following commands:

```
>>> OpenDatabase("elf_rs.vtk")      <– We open the database
>>> AddPlot("Pseudocolor","Re")      <– and select a plot type and a variable.
>>> DrawPlots()      <– Finally, we generate the plot.
```

This will produce a 3D pseudocolor image of the variable `Re` in the plotting window and we can interact with it as if we were using the graphical user interface. An example of the resulting plot is reported in Figure 1, where we used the step 0 of the scattering of a proton off a molecule of C2H4 calculated with the Octopus code.

[1] https://wci.llnl.gov/simulation/computer-codes/visit/executables

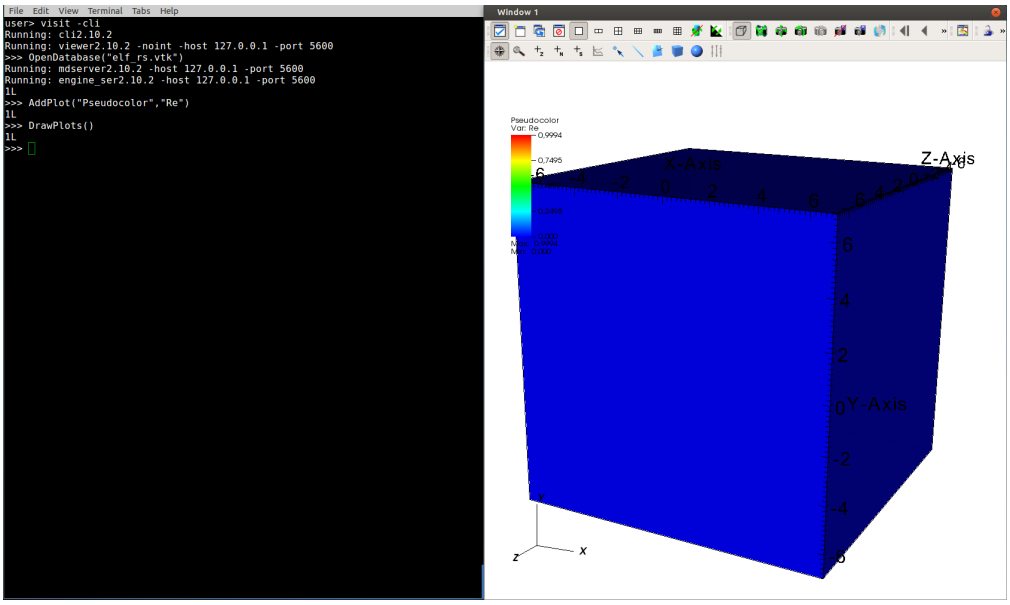


Figure 1. Pseudocolor plot of the input database.

From the image above, it is difficult to see the electron localization function around the molecule. Each pixel of the plot is colored according to the value of the corresponding cell in the input database and external cells are hiding the molecule located in the center of the box. We can solve this problem by introducing a slice operator, which corresponds to cutting a small slice of the 3D image and studying the electron localization function in a 2-dimensional plane.

```
>>> AddOperator("Slice", 0)      <– We add a slice to the current active plot
>>> DrawPlots()      <– and draw again the results.
```

Note that every time we add a new plot or a new operator, we need to run the `DrawPlots()` instruction again in order to update the plotting window.

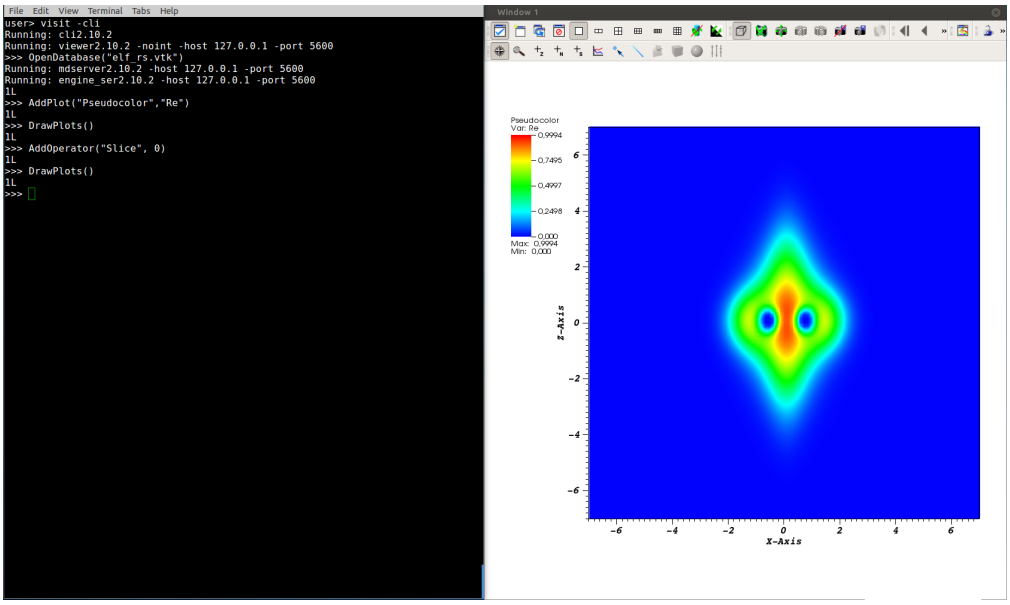


Figure 2. Pseudocolor plot after adding a slice operator.

By default, the slice is orthogonal to the Y-axis of the box and has an intercept equal to 0. In order to change the properties of the slice and adapt them to the input data, we can use the following commands:

```
>>> slice = SliceAttributes()      <– Retrieving the attributes of the slice operator.
>>> slice.originType = slice.Point      <– Centering the slice according to a specified point.
>>> slice.originPoint = (0, 0, 0.075)      <– Origin of the slice.
>>> slice.axisType = slice.ZAxis      <– Setting the slice perpendicular to the Z-axis.
>>> SetOperatorOptions(slice)      <– Setting the new attributes of the operator.
```

The corresponding output is shown below, in Figure 3.

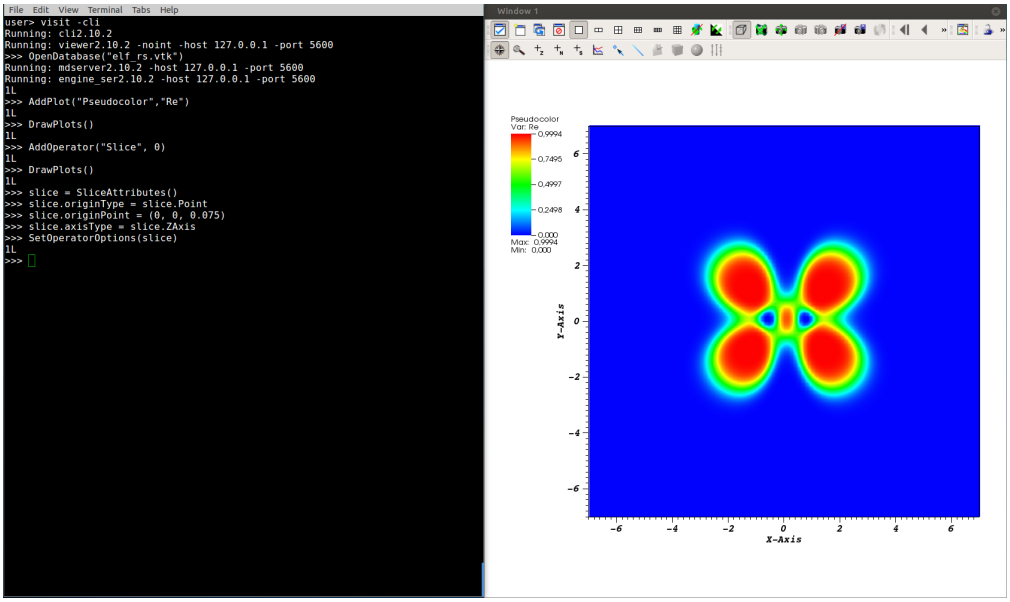


Figure 3. Setting different attributes for the slice operator.

Now that the operator has the correct attributes, we can change some of the properties of the plot. For example, we can select a different color table. Similarly to what we did for the slice operator, it is sufficient to retrieve the properties of the plot, modify their values, and set the new attributes:

```
>>> plot = PseudocolorAttributes()      <– Retrieving the plot properties
>>> plot.colorTableName = "hot_desaturated"      <– Changing the color table
>>> SetPlotOptions(plot)      <– Setting the new attributes of the plot.
```

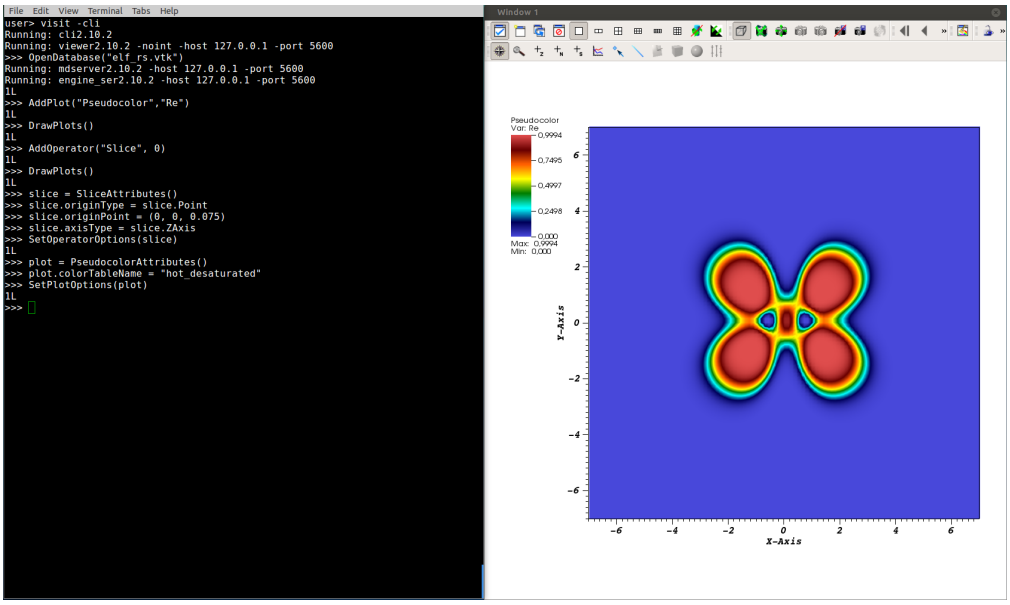


Figure 4. Pseudocolor plot with a different color table.

It's now time to set a better view for the image we want to produce. In the case of a 2D plot, we can do so by selecting the limits of the view and by extending the image to the full size of the plotting window:

```
>>> view = GetView2D()      <– Retrieving the properties of the 2D view.
>>> view.windowCoords = (-5, 5, -5, 5)      <– Setting the limits of the window.
>>> view.viewportCoords = (0, 1, 0, 1)      <– Setting the extent of the window.
>>> view.fullFrameActivationMode = view.On      <– Stretching the data to fill the screen.
>>> SetView2D(view)      <– Setting the new view values.
```

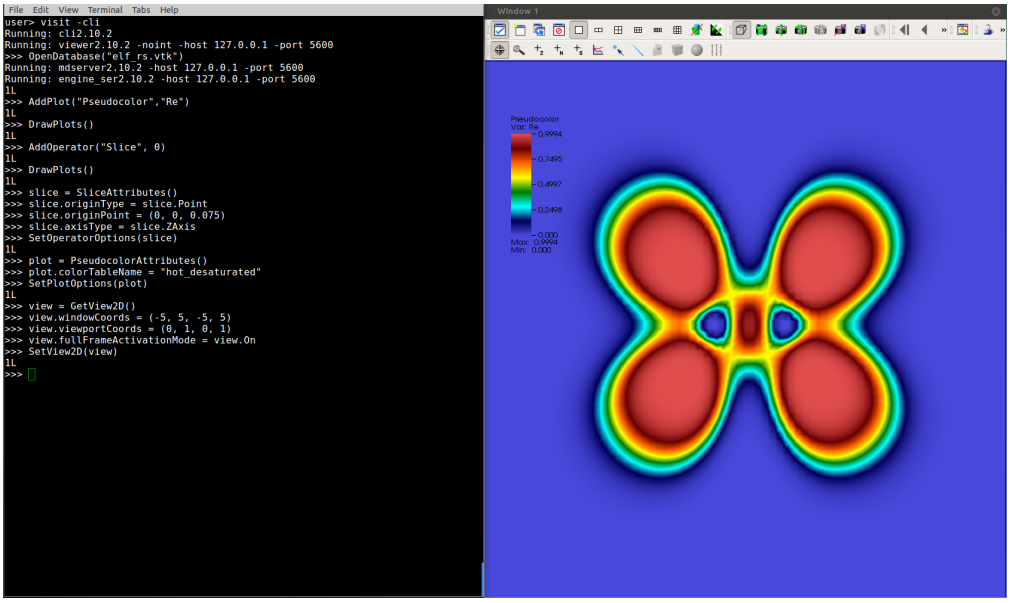


Figure 5. Pseudocolor plot after changing the view properties.

The position and the values of the color scale legend used by VisIt are usually good enough for a quick inspection of the data and for test images. When producing images for a publication, a poster or other

presentations, we may want to have more control on the size, position and values shown on the color scale legend. In this case, we must first find the name used by VisIt to identify the object "legend" and than we can modify its properties as usual. Note that the modifications are implemented as soon as we execute a command line and it is not necessary to set the new values with a `Set` command:

```
>>> legend_name = GetAnnotationObjectNames()[0]     <– Asking for the name of the legend.
>>> legend = GetAnnotationObject(legend_name)       <– Retrieving the legend's options.
>>> legend.managePosition = 0      <– Unlocking the position.
>>> legend.position = (0.25, 0.98)      <– Selecting a new position
>>> legend.orientation = legend.HorizontalBottom      <– and a different style.
>>> legend.xScale = 2      <– Stretching the legend along the X-axis
>>> legend.yScale = 0.5      <– and compressing along the Y-axis.
>>> legend.drawMinMax = 0      <– Removing min and max labels.
>>> legend.drawTitle = 0      <– Removing title label.
>>> legend.numTicks = 6      <– Number of ticks.
>>> legend.suppliedValues = (0, 0.2, 0.4, 0.6, 0.8, 1.)      <– Value of each tick.
>>> legend.fontHeight = 0.05      <– Increasing the size of the fonts
>>> legend.numberFormat = "%# -9.2f"      <– and choosing a different format for the numbers.
```

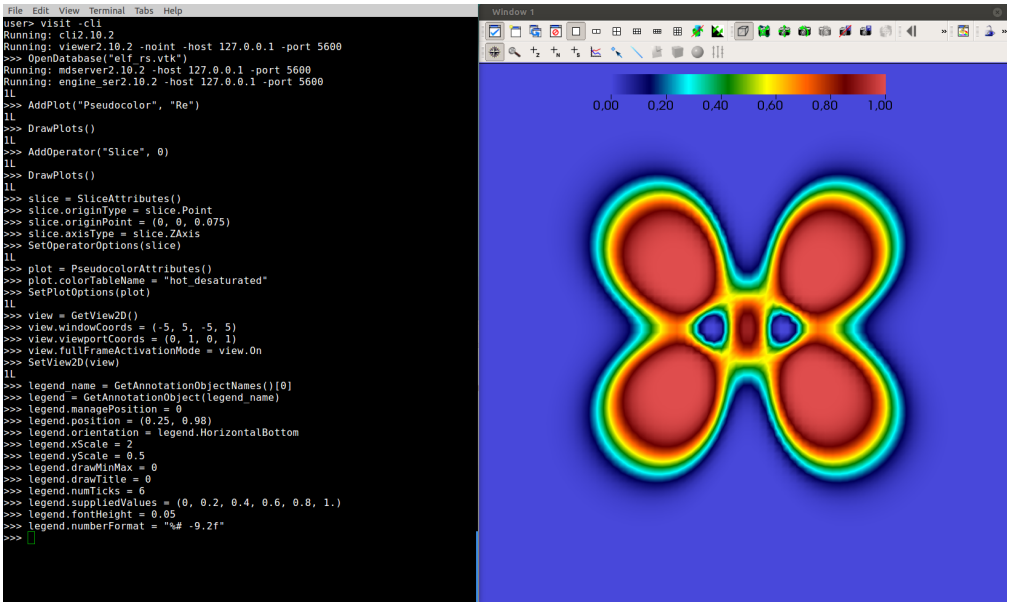


Figure 6. Changing the properties of the legend.

A lot of information is displayed by default in the plotting window. If we want to produce a movie using several frames extracted from the data, we may be interested in removing the color scale legend and the information about the user and the database. This can be done by modifying the annotations of the plotting window as follows:

```
>>> annot = AnnotationAttributes()      <– Retrieving the annotations.
>>> annot.userInfoFlag = 0      <– Removing the information on the user,
>>> annot.databaseInfoFlag = 0      <– on the database
>>> annot.legendInfoFlag = 0      <– and the color scale legend.
>>> annot.axes2D.visible = 0      <– Removing the axes from the plot.
>>> SetAnnotationAttributes(annot)      <– Setting the new annotation values.
```

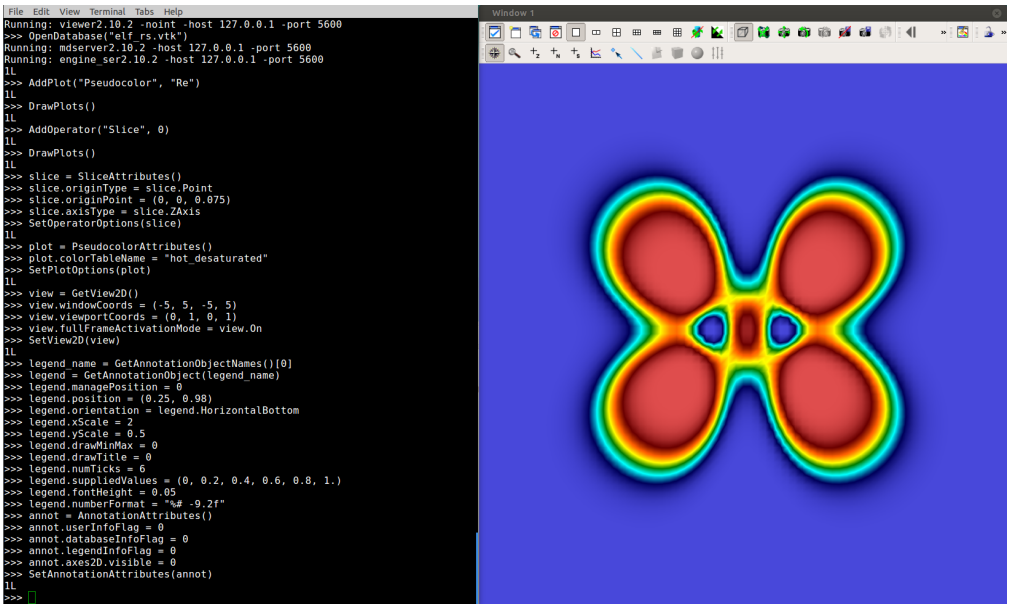


Figure 7. Final plot with annotations removed.

Finally, we save the image containing the current plot, we clean the visualization window and close the open database before quitting the VisIt's CLI.

```
>>> SaveWindow()      <– Saving the plotting windows with default attributes.
>>> DeleteAllPlots()      <– Deleting all plots and operators in the current window.
>>> CloseDatabase("elf_rs.vtk")      <– Closing the database.
>>> quit()      <– Closing VisIt's CLI.
```

Using the scripts

As shown in the previous section, the VisIt's command line interface provides instructions for the different options available through the graphical user interface. When generating frames for a movie and in all the cases where there are many output files to visualize, the CLI interface is usually a better choice with respect to the GUI interface. Using command line instructions is generally faster, less prone to errors and produces images with the same exact characteristics.

Especially when the number of command lines becomes significant, it is favorable to incorporate the instructions into a single script file that can be provided to the VisIt software using the `-s script.py` option:

```
> visit -cli -nowin -s script.py
```

or using

```
> visit -cli -s script.py
```

if we prefer to have a visualization window showing the output of the commands while the script is running.

In the remainder of this tutorial, we are going to describe 3 different scripts for the analysis of some outputs produced with the Octopus code:

1. *C2H4_2D.py* : will produce a set of 2-dimensional images for the scattering of a proton off a C2H4 molecule. The result will be similar to the plot obtained in the previous section, but will be extended to several input files.
2. *C2H4_3D.py* : this script is similar to the previous one, but explores the scattering process in a 3-dimensional space, introducing camera movements around the molecule and different types of 3D plots.
3. *azulene.py* : in this script, we follow the effect of a laser pulse on a molecule of azulene (C10H8). Given the complexity of the physical process and the multi-scale nature of the problem, we will use several plotting windows and we will combine them to create a single mosaic that illustrates the event from different points of view. In this script, we will create new variables combining available ones, and use queries to extract numerical information from the input database.

The initial part of all the scripts is very similar and is related to the location of the input files for VisIt. Using *C2H4_2D.py* as an example, the script will start with the following selection:

```
*** Welcome to C2H4_2D.py, a visualization script for VisIt ***

Description:  This script creates a set of 2D plots of the scatter
              of a proton off a molecule of C2H4.

Enter your choice:
1- Use default value for the setup variables
2- Specify the location of the file containing the list of inputs
3- Specify single inputs
```

where we are required to specify a numerical value in the range `1` to `3`.
By selecting `1`, the default values implemented into the script will be used and the code will look for an input text file in a pre-defined directory (see below). This will likely generate an error, if the script has not been modified before. Nevertheless, this option can be useful for users with a little bit of experience that want to modify the scripts and run tests or multiple visualizations of the same set of data.

Option number `2` allows us to indicate the location of a text file (called, for example, `inputdata.txt`) that contains the full path of the inputs we want to analyze. Each line of the text file should contain the location of a single input for VisIt, for example:

```
> more inputdata.txt
/scratch/user/output_iter/td.0000000/
/scratch/user/output_iter/td.0000015/
/scratch/user/output_iter/td.0000030/
/scratch/user/output_iter/td.0000045/
/scratch/user/output_iter/td.0000060/
...
```

Contrary to the previous options, option number `3` does not require any text file containing the location of the inputs, but it requires that we provide the directory where the inputs are located (in the example above this would be `/scratch/user/output_iter/`), the name of the first (`td.0000000/`) and last (`td.0000060/`) input and the step between the different input. For example, if the input files are the ones reported above, the step will be equal to `15`. In case the input files are not equally spaced and a constant step between the inputs can not be defined, it is recommended to create a text file with the list of inputs and use option `2`.

In all the scripts the following options are available:

- Select an output directory.

- Specify a name for the output images. Note that the final name of the image will take the `.png` file extension and will include a 4-digit number after the selected name.
- Possibility to create a video using the images just produced. Note that, in order to create a video, the `ffmpeg` package must be available (see section *Setting up the environment*). When creating a video, you can select a value between `1` and `100` that specifies the movie's speed. In practice, this number defines how many times each frame is used when creating the video.
  It is also possible to create a video with images already available in the output folder. In order to do so, it is sufficient to provide an empty list of input file to the scripts.

Other plot options specific of each script may be available and are described in the following sections. There are also some parameters that are hard-coded into the scripts, but may be changed if necessary. These parameters control:

- The default directories for the input text file (`inputdata`) and for the output (`outdir`).
- The creation of molecular bonds. By setting `bonds=True` the molecular bonds will be displayed in the images.
- The name of the database and the name of the variables included in the database. Depending on the specific script, these variables may have different names. Please, refer to comments at the beginning of the script for additional information.
- When available, if the script should use the `.cube` database instead of the default `.vtk` file. This option can be activated by using `usecube=True`.

Once all parameters have been specified, the scripts will check if the input files exist and will eventually remove from the internal list of inputs those files that are not present. If the internal list of inputs is empty, the scripts will skip to the movie creation section and will use the images already available in the output directory for the creation of the movie.

As a last check before starting producing the requested images, all the scripts will provide a summary of the setup variables. Moreover, at runtime, the scripts will print on screen an estimate of the time left before the completion of all the images. This estimate is calculated using the average time spent to produce the images that have been already completed. Therefore, especially during the first steps, it may not be very accurate, but after a while, it should provide a good estimate of the number of seconds still necessary in order to analyze all inputs.

In the following sections, we will look at the specific options of each script.

2D plot case: *C2H4_2D.py*

As previously mentioned, the *C2H4_2D.py* script creates a set of 2-dimensional images of the scattering of a proton off a C2H4 molecule. The images are obtained extracting a slice from each input file along the same plane, adding the contours corresponding to an electron localization function of `0.8` and over-plotting the position of the elements.

In addition to the previous options, this script requires as input:

- The plot limits: provided as `xmin`, `xmax`, `ymin` and `ymax`. These values set the plotting range.
- The color table. The color table adopted by default is hard-coded into the script, but it can be changed to one of the several color tables provided by VisIt during run time.

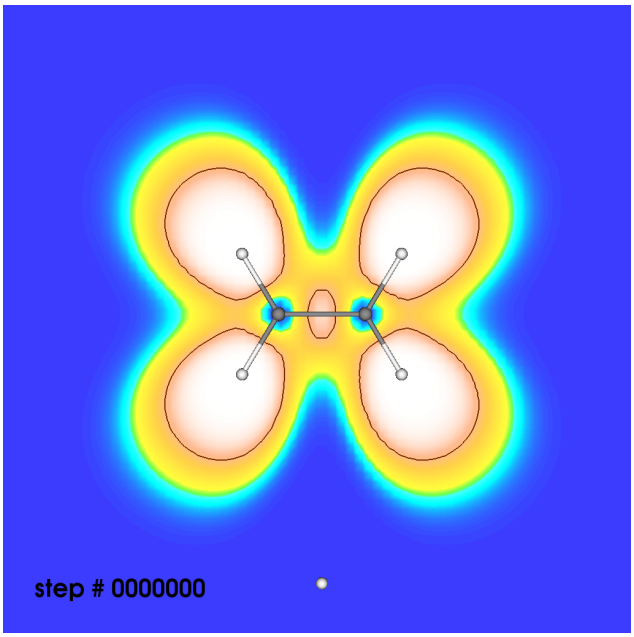


Figure 8. Example of a 2D frame of the scattering of a proton off a molecule of C2H4. Note that the molecular bonds are not plotted by default.

3D plot case: *C2H4_3D.py*

The script *C2H4_3D.py* adds a level of complexity to the previous one by shifting the visualization to the 3-dimensional space. In this case, it is possible to set the following parameters:

- Zoom value. Similarly to the plot limits, the zoom value set the apparent distance between the camera and the molecule, therefore determining the field of view of the images.
- Type of plot. VisIt offers several different types of plot for 3D imaging. In this script, we focus on volume rendering and contour plots:

    - Volume rendering is a visualization technique where parts of the data are transparent or opaque, depending on a volume transfer function, which maps data values to colors and opacities. In this script, we consider only 3D texturing (simply called *volume plot* in the script) and ray casting. This second method is much slower than the first one, but produces images of superior quality. See this link[2] for additional information on volume rendering. The color table for these plots can be specified during run time, similarly to the 2-dimensional case.
    - Contour plots (also known as isosurfaces) are 3-dimensional surfaces characterized by the same value of the plotted variable. The number, color and opacity of the different contours can be modified by changing the value of the variables `contval` and `colval` inside the script.

- Background grid. It is possible to add a background grid to the plot in order to have some reference points in space during the scattering process.
- Initial rotation around the molecule. This option activates a rotation of the camera point around the molecule for the first input file. The rotation is performed interpolating the position of the camera between selected points in space and, with minor modification to the function `rotatecamera()` in the script, it can be easily adapted to more complicated camera movements.

[2] http://www.visitusers.org/index.php?title=Volume_Rendering

- Camera rotation during the scattering. With this option, it is possible to observe the scattering process while the camera is rotating around the molecule. The camera will complete a single rotation between the first and the last input file.

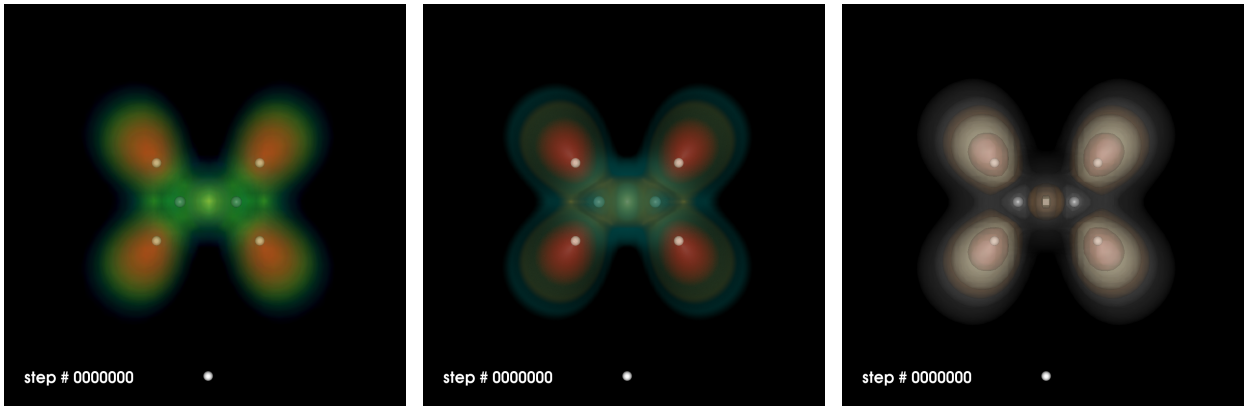


Figure 9. Example of 3D frames of the scattering of a proton off a molecule of C2H4. From left to right: volume plot with 3D texturing, volume plot with ray casting and multiple contour plot.

Multi-window plot case: *azulene.py*

For some complex situations, a single plotting window may not be sufficient to efficiently illustrate all the aspects of the problem. Therefore, it may be necessary to have several plots, each focusing on a particular feature of the physical process. This is the case, for example, of a laser pulse propagating towards a molecule of azulene. Although VisIt can handle up to 16 simultaneous plotting windows, in this script we will create a mosaic using only 4 different panels in order to follow the propagation of the laser pulse and its effects on the azulene molecule. An example of the resulting images is reported in Figure 10, where the top panels display the intensity of the electric field along the Y-Z plane and the Z axis, while the bottom panels provide a 3D illustration of the problem and the effect of the electric field on the plane of the molecule.

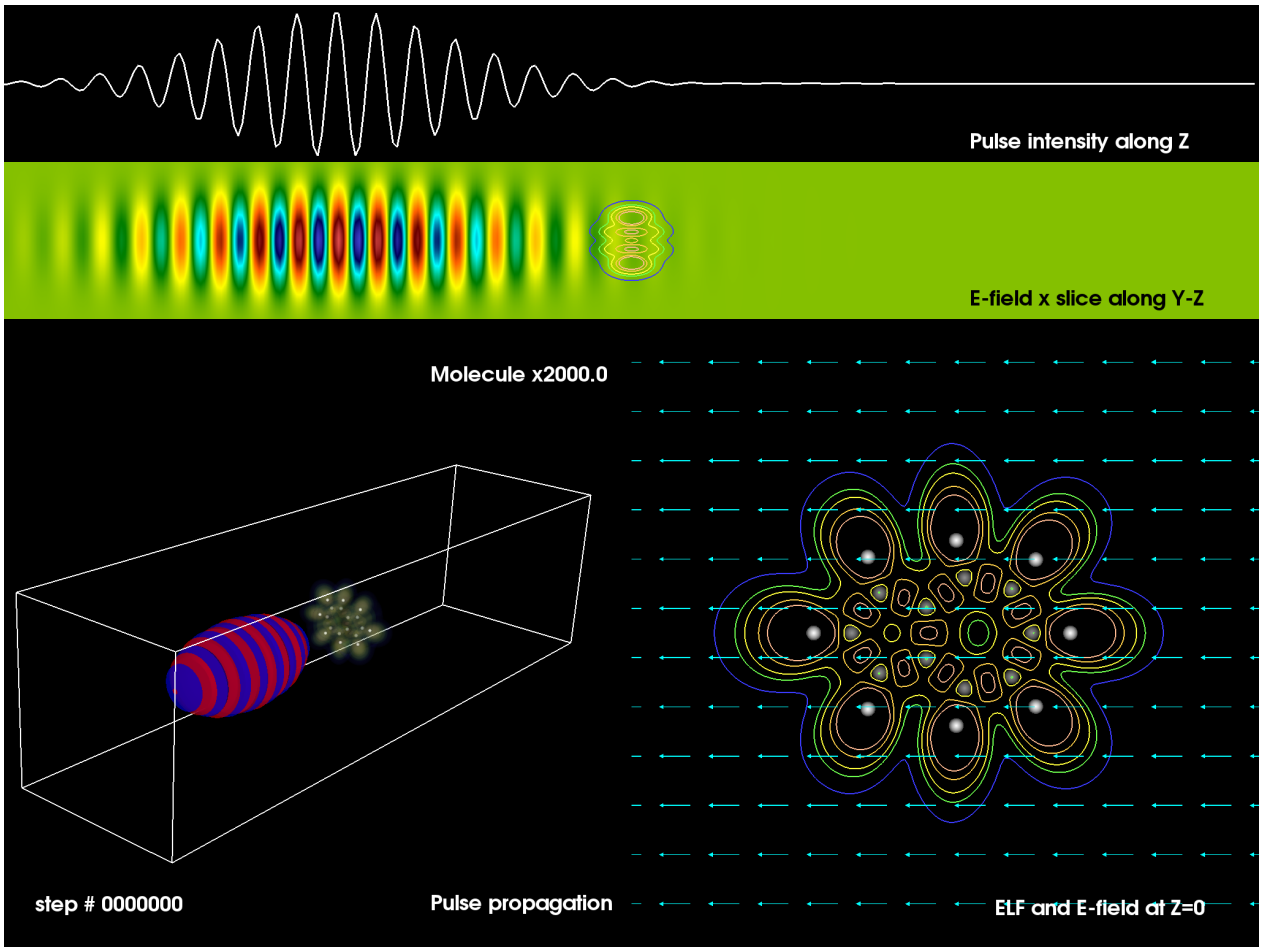


Figure 10. Example of a mosaic of four images generated in the case of a laser pulse propagating towards a molecule of azulene.

In this script, only a single parameter can be selected in addition to the standard options:

- Zoom for the molecule. The typical scale of a laser pulse and a molecule can be very different. This parameter rescales the size of the molecule, so that it is comparable with the length scale of the electric field and they can both be reproduced on the same plot. If the value of the zoom is larger than `1`, then a set of images for the first input is created, where the molecule is gradually rescaled to the selected value. Note that in the *Pulse propagation* window, the size of the canvas is associated with the length scale of the electric field. Therefore, the molecule may not be visible if the zoom is set to a low value. On the other hand, the size of the *ELF and E-field at Z=0* window is associated with the size of the molecule and the arrows representing the electric field may have a different length, depending on the value of the zoom.

As a final note, some of the calculations performed by this script are executed in an additional window (indicated as *Window 5*) that is created, manipulated and finally destroyed at each time step.

Resources

For additional information, useful tips and examples about VisIt and the command line interface, please refer to:

- VisIt at MPCDF:
  http://www.mpcdf.mpg.de/services/visualization/software-collection/visit
- VisIt Users:
  http://visitusers.org/index.php?title=Main_Page
- VisIt Python Interface Manual:
  http://visit.ilight.com/svn/visit/trunk/releases/2.10.0/VisItPythonManual.pdf
- Script examples:
  http://www.visitusers.org/index.php?title=Python_examples